

FOODRESCUE — YELLOWPAPER

# Technical specification

of the Solana program and the verification protocol

Program v1

TradeState v2

RescueProofState v2

ProtocolConfig v1

IN THIS DOCUMENT

# Contents

|           |                                        |  |
|-----------|----------------------------------------|--|
| <b>1</b>  | <b>Conventions</b>                     |  |
| <b>2</b>  | <b>Program identity</b>                |  |
| <b>3</b>  | <b>Account model</b>                   |  |
| <b>4</b>  | <b>State layout</b>                    |  |
| <b>5</b>  | <b>Instruction set</b>                 |  |
| <b>6</b>  | <b>On-chain state machine</b>          |  |
| <b>7</b>  | <b>Fee arithmetic and distribution</b> |  |
| <b>8</b>  | <b>Program invariants</b>              |  |
| <b>9</b>  | <b>Off-chain verification protocol</b> |  |
| <b>10</b> | <b>Wallet ownership protocol</b>       |  |
| <b>11</b> | <b>Error codes</b>                     |  |
| <b>12</b> | <b>Test vectors</b>                    |  |

## SECTION 1

# Conventions

| CONVENTION                          | VALUE                                                                       |
|-------------------------------------|-----------------------------------------------------------------------------|
| Integers                            | little-endian, no padding, no alignment                                     |
| <code>u64</code> / <code>i64</code> | 8 bytes                                                                     |
| <code>Pubkey</code>                 | 32 raw bytes; base58 only at the JSON boundary                              |
| Hash                                | SHA-256, 32 bytes                                                           |
| Serialisation                       | manual, byte by byte — <b>no Borsh, no Anchor, no 8-byte discriminators</b> |
| Instruction discriminator           | 1 byte ( <code>data[0]</code> ), followed by a fixed-size payload           |
| Timestamps                          | <code>i64</code> Unix, from the <code>Clock</code> sysvar                   |
| RPC commitment                      | <code>confirmed</code> (configurable)                                       |

The absence of Anchor is deliberate: the layout is public, stable and reproducible in any language without an IDL. The PHP backend decodes the same bytes the Rust writes, and there is a [cross-compatibility test](#) that proves it.

SECTION 2

# Program identity

| DEVNET            |                                              |
|-------------------|----------------------------------------------|
| Program ID        | Ex6CN32gBUH2JALUwbqyn8ZMwWmBNrrHa4sjDNMAm5sd |
| FRUSD mint        | 9tVPExJFkBU3yLgyo8fVzFVQj2t2boEESSpmoYikfxRr |
| SPL Token Program | TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA  |
| System Program    | 11111111111111111111111111111111             |
| Fee               | PROTOCOL_FEE_BPS = 200 (2.00%)               |

Dependencies: solana-program 2.2, spl-token 8.0 (no-entryptoint ). Roughly 1,370 lines of Rust in total.

## SECTION 3

# Account model

Four account types, all PDAs — none of them has a private key.

| ACCOUNT               | SEEDS                                                      | OWNER        | SIZE  |
|-----------------------|------------------------------------------------------------|--------------|-------|
| <b>ProtocolConfig</b> | "foodrescue_protocol"    authority                         | program      | 98 B  |
| <b>Trade</b>          | "foodrescue_trade"    trade_id: u64 LE    buyer            | program      | 244 B |
| <b>Vault</b>          | "foodrescue_vault"    trade_id: u64 LE    buyer            | SPL<br>Token | 165 B |
| <b>RescueProof</b>    | "foodrescue_rescue"    trade_id: u64 LE    ngo    producer | program      | 147 B |

**Why the buyer's wallet is part of the Trade and Vault seeds.** The `trade_id` comes from the database and is sequential, therefore predictable. Were it the only seed, a third party could occupy the address ahead of the legitimate buyer and jam the trade. With `buyer` in the derivation, only whoever controls that wallet can create that PDA — address front-running ceases to exist.

The same reasoning applies to RescueProof, which includes **both** attesting wallets.

**The Vault is an ordinary SPL Token account whose authority is the Trade PDA.** This is the core of the thesis: moving the balance requires the program to sign with the trade's seeds, which happens only inside `SettleTrade` or `CancelTrade`. No human — not even the platform admin — can produce that signature.

SECTION 4

# State layout

## 4.1 ProtocolConfig — 98 bytes, version = 1

| OFFSET | SIZE | FIELD     | TYPE   |
|--------|------|-----------|--------|
| 0      | 1    | version   | u8 = 1 |
| 1      | 1    | bump      | u8     |
| 2      | 32   | authority | Pubkey |
| 34     | 32   | treasury  | Pubkey |
| 66     | 32   | mint      | Pubkey |

## 4.2 TradeState — 244 bytes, version = 2

| OFFSET | SIZE | FIELD           | TYPE                        |
|--------|------|-----------------|-----------------------------|
| 0      | 1    | version         | u8 = 2                      |
| 1      | 1    | trade_bump      | u8                          |
| 2      | 1    | vault_bump      | u8                          |
| 3      | 8    | trade_id        | u64                         |
| 11     | 32   | buyer           | Pubkey                      |
| 43     | 32   | producer        | Pubkey                      |
| 75     | 32   | carrier         | Pubkey (zeros = no carrier) |
| 107    | 32   | mint            | Pubkey                      |
| 139    | 32   | vault           | Pubkey                      |
| 171    | 32   | protocol_config | Pubkey                      |
| 203    | 8    | product_amount  | u64                         |
| 211    | 8    | shipping_amount | u64                         |
| 219    | 8    | protocol_fee    | u64                         |
| 227    | 8    | total_amount    | u64                         |
| 235    | 8    | expires_at      | i64                         |
| 243    | 1    | status          | u8                          |

status: 0 Initialized · 1 Funded · 2 Settled · 3 Cancelled · 4 ReadyForPickup · 5 InTransit · 6 Delivered

The state carries **every** party and every amount. Nothing has to be cross-referenced against the database to know who receives how much — the PDA is self-sufficient and auditable by third parties.

4.3 RescueProofState — 147 bytes, version = 2

| OFFSET | SIZE | FIELD         | TYPE     |
|--------|------|---------------|----------|
| 0      | 1    | version       | u8 = 2   |
| 1      | 1    | bump          | u8       |
| 2      | 8    | trade_id      | u64      |
| 10     | 32   | producer      | Pubkey   |
| 42     | 32   | ngo           | Pubkey   |
| 74     | 32   | carrier       | Pubkey   |
| 106    | 32   | metadata_hash | [u8; 32] |
| 138    | 8    | created_at    | i64      |
| 146    | 1    | status        | u8       |

status: 0 PendingProducer · 1 Confirmed

metadata\_hash is the SHA-256 of the canonical JSON ( JSON\_UNESCAPED\_SLASHES | JSON\_UNESCAPED\_UNICODE ) of:

```
{ "version": 1, "trade_id": ..., "surplus_lot_id": ..., "product_id": ..., "quantity": "...",
  "unit": "...", "producer_id": ..., "ngo_id": ..., "carrier_id": ..., "shipping_amount": "..." }
```

## SECTION 5

# Instruction set

Ten instructions. `data[0]` is the tag; the rest is a fixed-size payload — **any other size is rejected**, which eliminates an entire class of deserialisation attacks.

| TAG | INSTRUCTION                     | PAYLOAD | TOTAL |
|-----|---------------------------------|---------|-------|
| 0   | <code>InitializeTrade</code>    | 104 B   | 105 B |
| 1   | <code>FundTrade</code>          | 0       | 1 B   |
| 2   | <code>InitializeProtocol</code> | 32 B    | 33 B  |
| 3   | <code>SettleTrade</code>        | 0       | 1 B   |
| 4   | <code>CancelTrade</code>        | 0       | 1 B   |
| 5   | <code>CreateRescueProof</code>  | 72 B    | 73 B  |
| 6   | <code>MarkReadyForPickup</code> | 0       | 1 B   |
| 7   | <code>ConfirmPickup</code>      | 0       | 1 B   |
| 8   | <code>MarkDelivered</code>      | 0       | 1 B   |
| 9   | <code>ConfirmRescueProof</code> | 0       | 1 B   |

## 5.1 `InitializeProtocol` — tag 2

Payload: `treasury: Pubkey` (32 B).

Accounts: `authority` (signer, writable) · `protocol_config` (writable) · `mint` · `system_program`

Creates the configuration PDA derived from the authority itself. Validates that the mint unpacks as an SPL `Mint`. One authority per configuration; the operation is idempotent by address because creation fails if the account already exists.

## 5.2 InitializeTrade — tag 0

| OFFSET | SIZE | FIELD                                  |
|--------|------|----------------------------------------|
| 0      | 8    | trade_id                               |
| 8      | 8    | product_amount                         |
| 16     | 8    | shipping_amount                        |
| 24     | 8    | protocol_fee                           |
| 32     | 8    | expires_at                             |
| 40     | 32   | producer                               |
| 72     | 32   | carrier (zeros if there is no freight) |

Accounts: `buyer` (signer, writable) · `trade_pda` (w) · `vault` (w) · `buyer_token` (w) · `protocol_config` · `mint` · `system_program` · `token_program`

### Preconditions checked by the program:

- `protocol_config` belongs to the program and its PDA re-derives from the stored authority
- `protocol.mint == mint`
- `trade_pda` and `vault` match `find_program_address` over the canonical seeds
- `producer != 0` and `producer != buyer` — nobody trades with themselves
- `shipping_amount > 0` → `carrier != 0`
- `protocol_fee == floor(product_amount × 200 / 10_000)`, otherwise `InvalidFee`
- `total_amount = product_amount + shipping_amount` via `checked_add`
- `total_amount != 0` and `expires_at > now`, otherwise `PaymentExpired`

**Effects:** creates the Trade PDA (244 B, owner = program), creates the Vault (165 B, owner = SPL Token), initialises the Vault via `initialize_account3` with authority = Trade PDA, and writes the state with `status = 0`.

## 5.3 FundTrade — tag 1

Accounts: `buyer` (signer) · `trade_pda` (w) · `buyer_token` (w) · `vault` (w) · `mint` · `token_program`

Requires `status == Initialized`, `state.buyer == buyer`, `now < expires_at`, and that both token accounts unpack with the correct owner and mint. Re-derives Trade and Vault with `create_program_address` from the bumps **stored in the state** — it does not trust what arrived in the transaction.

Moves `total_amount` from `buyer_token` to the Vault via `transfer_checked` (which validates decimals) and advances to `status = 1`.

## 5.4 `SettleTrade` — tag 3

Accounts: `buyer (signer) · trade_pda (w) · vault (w) · buyer_token (w) · protocol_config · producer_token (w) · treasury_token (w) · mint · token_program · carrier_token (w)` — only if `shipping_amount > 0`

Requires `status == Delivered` and `escrow.amount >= total_amount`. Checks the **owner** of every destination account against the state: `producer_token.owner == state.producer`, `treasury_token.owner == protocol.treasury`, `buyer_token.owner == state.buyer`, `carrier_token.owner == state.carrier`. Forged token addresses do not pass.

Distributes in a single transaction, signing with the Trade PDA seeds:

```
producer_token  ← product_amount - protocol_fee    (if > 0)
treasury_token  ← protocol_fee                    (if > 0)
carrier_token   ← shipping_amount                 (if > 0)
buyer_token     ← escrow.amount - total_amount     (surplus, if > 0)
```

Then `status = 2`. Returning the surplus exists because **anyone can transfer tokens into an SPL account**: without it, an unsolicited deposit would sit trapped in the vault forever.

## 5.5 `CancelTrade` — tag 4

Accounts: `buyer · producer · trade_pda (w) · vault (w) · buyer_token (w) · mint · token_program`

Only in `status ∈ {Initialized, Funded}`. The required signatures depend on the state:

| STATE                        | SIGNATURES                |
|------------------------------|---------------------------|
| Initialized (empty vault)    | buyer <b>or</b> producer  |
| Funded (vault holding money) | buyer <b>and</b> producer |

This is the most important rule of the protocol after the escrow itself: once the money is locked, **neither party unwinds the deal alone**.

Checks `escrow.amount >= (total_amount if Funded, else 0)` and returns the vault's **entire balance** — not just `total_amount` — for the same reason as the settlement surplus: an unsolicited transfer must not be able to block a cancellation. Then `status = 3`.

## 5.6 `MarkReadyForPickup` / `ConfirmPickup` / `MarkDelivered` — tags 6, 7, 8

Accounts: `actor` (signer) · `trade_pda` (writable). That is all — these are pure state transitions, with no token movement.

| TAG | TRANSITION                                           | WHO SIGNS                                                                             |
|-----|------------------------------------------------------|---------------------------------------------------------------------------------------|
| 6   | <code>Funded</code> → <code>ReadyForPickup</code>    | <code>state.producer</code>                                                           |
| 7   | <code>ReadyForPickup</code> → <code>InTransit</code> | <code>state.carrier</code> , or <code>state.buyer</code> if <code>carrier == 0</code> |
| 8   | <code>InTransit</code> → <code>Delivered</code>      | <code>state.buyer</code> (the recipient)                                              |

Delivery is always confirmed by the **recipient**, never by whoever carries the goods — the party with an interest in attesting that it received is the same one who unlocks the payment.

## 5.7 `CreateRescueProof` — tag 5

| OFFSET | SIZE | FIELD                      |
|--------|------|----------------------------|
| 0      | 8    | <code>trade_id</code>      |
| 8      | 32   | <code>carrier</code>       |
| 40     | 32   | <code>metadata_hash</code> |

Accounts: `ngo` (signer, writable) · `producer` · `protocol_config` · `rescue_pda` (w) · `system_program`

The producer **does not sign here** — it appears only as a key in the seeds and in the state. Rejects `ngo == producer` and `metadata_hash == 0`. Creates the PDA with `status = 0` (pending the producer).

## 5.8 `ConfirmRescueProof` — tag 9

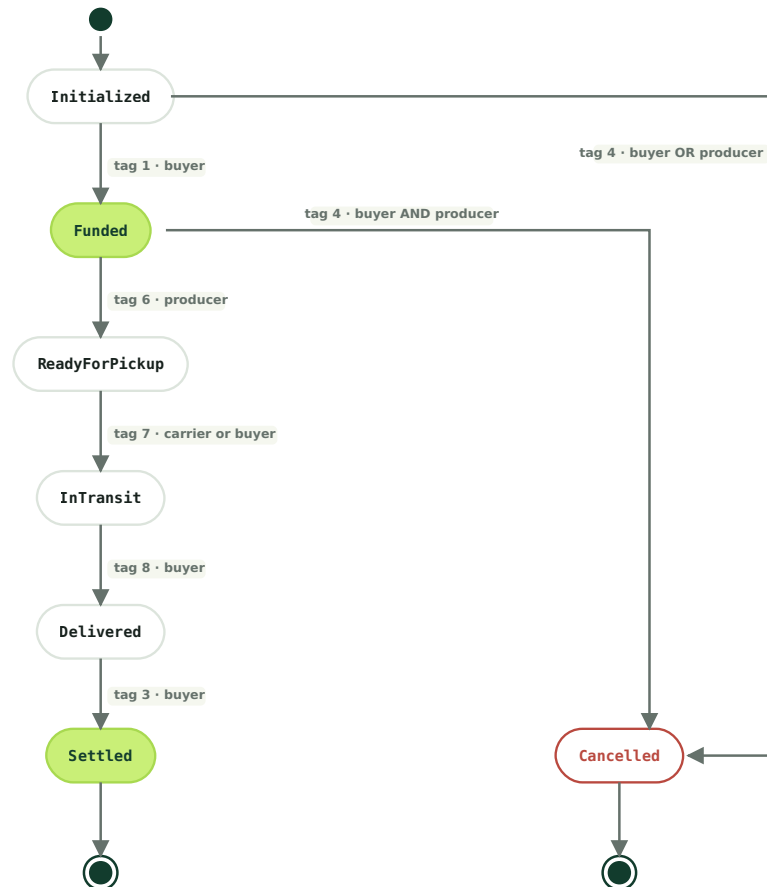
Accounts: `producer` (signer) · `rescue_pda` (writable)

Requires `status == PendingProducer` and `state.producer == producer`. Re-derives the PDA with the stored bump. Advances to `status = 1`.

Two transactions rather than one carrying two signatures, because the parties sign at different moments and on different machines, and a shared blockhash would expire along the way.

## SECTION 6

# On-chain state machine



Transitions absent from the diagram are impossible: every instruction compares `state.status` against the exact expected value and returns `InvalidState` in any other case. There is no backward transition, no path from `Delivered` to `Cancelled`, and `Settled/Cancelled` are terminal.

## SECTION 7

# Fee arithmetic and distribution

```
pub const PROTOCOL_FEE_BPS: u64 = 200;

pub fn protocol_fee(product_amount: u64) -> u64 {
    ((product_amount as u128 * PROTOCOL_FEE_BPS as u128) / 10_000) as u64
}
```

The multiplication is widened to `u128` before dividing — no overflow is possible for any `u64`, and rounding is **always downward, in the producer's favour**. The value arrives in the instruction from the backend, but the program recomputes it and rejects any divergence with `InvalidFee`: the fee is not negotiable at the edge.

Sums use `checked_add / checked_sub` with `ArithmeticOverflow`. The backend performs the same calculation with `brick/math` over decimals and converts to the mint's base units, checking that the declared precision can hold the value — a `product_amount` with more decimal places than the mint supports is refused before it ever reaches the chain.

| PARTY           | FORMULA                                                   |
|-----------------|-----------------------------------------------------------|
| Producer        | <code>product_amount - protocol_fee</code>                |
| Treasury        | <code>protocol_fee</code>                                 |
| Carrier         | <code>shipping_amount</code>                              |
| Buyer (surplus) | <code>escrow.amount - total_amount</code>                 |
| <b>Sum</b>      | <code>escrow.amount</code> — the vault empties completely |

## SECTION 8

# Program invariants

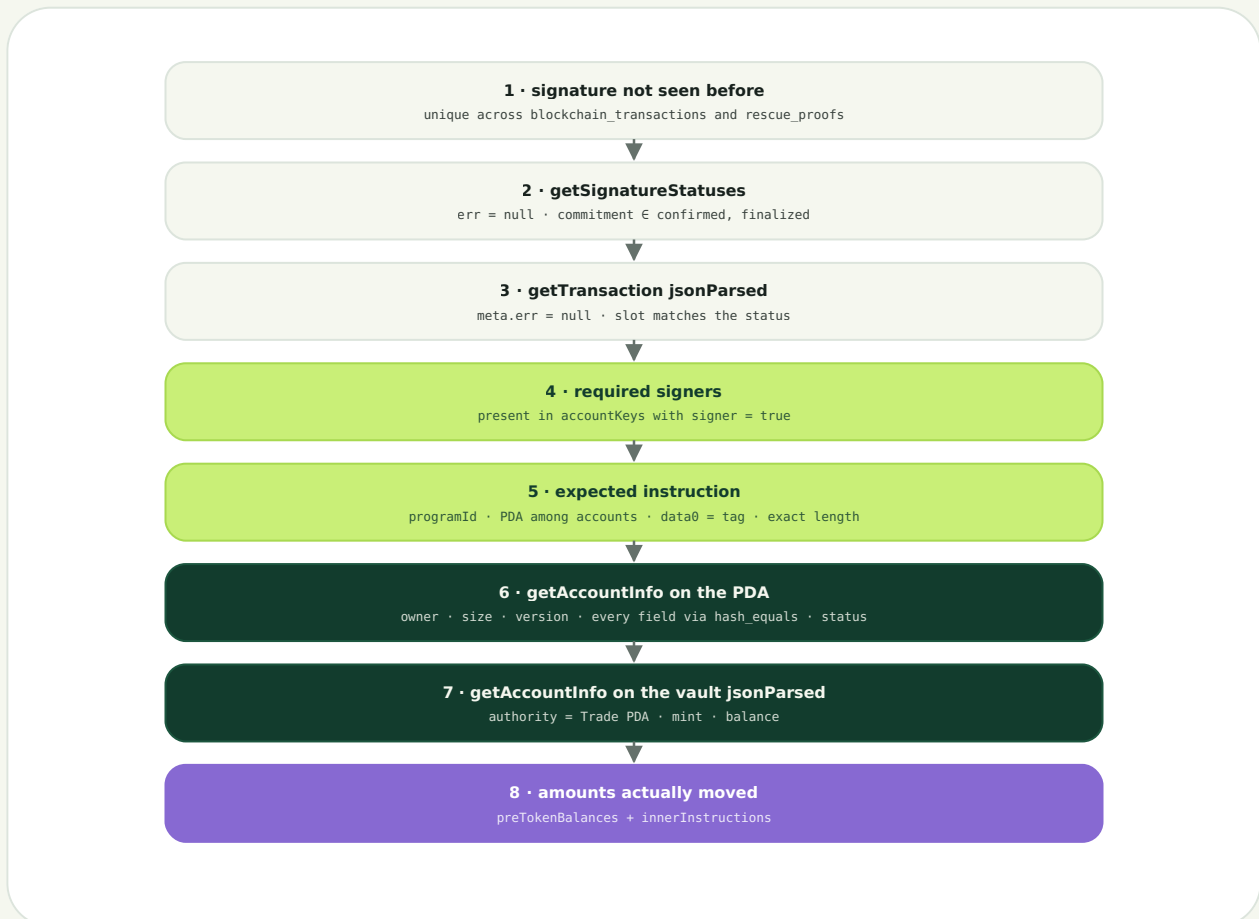
Properties the program maintains under any sequence of instructions:

1. **The vault only moves inside `SettleTrade` or `CancelTrade`.** The authority is the Trade PDA, and the seed signature is produced only along those two paths.
2. **Every PDA derivation is re-verified** — at creation with `find_program_address`, later with `create_program_address` from the bumps stored in the state. No account passed in the transaction is accepted for the address it claims to hold.
3. **Every token account has its owner checked against the state**, never against the input.
4. **The distributed sum is exactly the vault balance.** No dust is left over and no value stays trapped.
5. **`total_amount` never changes after `Initialize`** — it is written once and only read afterwards.
6. **An unsolicited deposit jams nothing**: the surplus returns on settlement, the entire balance returns on cancellation.
7. **No transition skips a state.** `from` is compared for exact equality.
8. **Deadlines come from the chain, not the client**: `expires_at` is compared against the `Clock` sysvar.

## SECTION 9

# Off-chain verification protocol

The backend never believes a signature simply because it was presented. Every confirmation goes through this sequence before any database write — and all of it happens inside a transaction holding `SELECT ... FOR UPDATE` over the trade.



**Step 5 — expected instruction.** The verifier walks the transaction's instructions looking for one whose `programId` matches, with the PDA in its account list, `data[0] == tag` and a length **exactly** equal to the table in section 5. A transaction that executes the wrong instruction, or the right one over a different PDA, does not pass.

**Step 6 — re-reading the state.** Compares field by field against what was promised at preparation: `trade_id`, `buyer`, `producer`, `carrier`, `mint`, `vault`, `protocol_config` (keys compared with `hash_equals` over the raw bytes), the four monetary amounts, the `expires_at` and the `status`, which must be exactly the one expected for that trade. A version other than 2, or a size other than 244, is refused before any field is read.

**Step 8 — the money that actually moved.** Reading the final state alone is not enough to know how much was transferred. On settlement, the backend locates the vault in `meta.preTokenBalances` (matching `accountIndex`, `mint` and `owner ==`

`trade_pda`) and requires a prior balance  $\geq$  `total_amount`; the difference is recorded as a surplus refund. On cancellation it goes deeper: it finds the cancellation instruction, reads the `innerInstructions` at that index, sums every `transferChecked` whose `source` is the vault and whose `authority` is the Trade PDA, and requires that sum to cover the prior balance. Tokens that arrived earlier in the same transaction do not escape the check.

**RPC failures never become success.** A network error or a malformed response yields a `502`, never a state advance.

## SECTION 10

# Wallet ownership protocol

Binding a wallet to an account requires proving possession of the private key by an Ed25519 signature over a server-issued message.

## FoodRescue Wallet Verification

Version: 1

Purpose: <registration | verify | surplus\_publication>

Wallet: <base58>

User ID: <id> ← absent for the registration purpose

Nonce: <uuid v4>

Issued At: <ISO-8601 UTC>

Expires At: <ISO-8601 UTC>

| PROPERTY     | RULE                                                                       |
|--------------|----------------------------------------------------------------------------|
| Verification | <code>sodium_crypto_sign_verify_detached(sig, message, pubkey)</code>      |
| Signature    | 64 bytes, carried in base64                                                |
| Public key   | 32 bytes, decoded from base58                                              |
| TTL          | default 5 min, bounded to <code>[1, 30]</code>                             |
| Use          | <b>single</b> — <code>used_at</code> written under <code>FOR UPDATE</code> |
| Scope        | purpose, wallet and <code>user_id</code> checked together                  |

The purpose is part of the signed message, so a signature obtained to change wallets does not work for publishing a surplus. When a new wallet is verified, all of that user's pending challenges are invalidated at once, and the uniqueness of `solana_wallet_address` is guaranteed by a unique index in the database on top of the check in code.

Publishing a surplus consumes a challenge with the `surplus_publication` purpose: the lot in the catalogue carries the cryptographic commitment of whoever published it.

## SECTION 11

# Error codes

Returned as `ProgramError::Custom(n)`.

| N | ERROR                            | MEANING                                                               |
|---|----------------------------------|-----------------------------------------------------------------------|
| 0 | <code>InvalidInstruction</code>  | Unknown tag, or payload of the wrong size                             |
| 1 | <code>InvalidPda</code>          | Address does not match the canonical derivation                       |
| 2 | <code>InvalidAccount</code>      | Wrong signer, write permission or program                             |
| 3 | <code>InvalidState</code>        | State version or <code>status</code> incompatible with the transition |
| 4 | <code>InvalidFee</code>          | Fee diverges from <code>floor(product × 200 / 10_000)</code>          |
| 5 | <code>PaymentExpired</code>      | Deadline passed, or zero total at initialisation                      |
| 6 | <code>ArithmeticOverflow</code>  | <code>checked_add</code> / <code>checked_sub</code> failed            |
| 7 | <code>InvalidTokenAccount</code> | Incompatible owner, mint or balance on an SPL account                 |

## SECTION 12

# Test vectors

The binary layout is verified from both ends by fixtures **generated by the Rust and read by the PHP**.

`solana/examples/export_layout.rs` serialises the three states with known values and writes `solana/fixtures/{trade,protocol,rescue}.bin`. The same files live in `www/tests/Fixtures/Solana/`, and `RustLayoutCompatibilityTest` asserts that the PHP decoder extracts exactly the values the Rust wrote:

```
trade.bin      244 B  trade_id 42 · bumps 254/253 · product 100,123,456
                  freight 12,345,678 · fee 2,002,469 · total 112,469,134
                  expires_at 2_000_000_000 · status 1 (Funded)
protocol.bin   98 B   version 1 · bump 250
rescue.bin     147 B  version 2
```

A layout change on one side breaks the test on the other. That is what stops this specification and the code from silently drifting apart.

| SUITE          | WHERE                                                                                |
|----------------|--------------------------------------------------------------------------------------|
| Solana program | <code>cd solana &amp;&amp; cargo test (tests/program.rs, tests/validation.rs)</code> |
| Backend        | <code>cd www &amp;&amp; composer test:php</code>                                     |
| Front end      | <code>cd www &amp;&amp; npm test</code>                                              |
| Devnet E2E     | <code>cd solana &amp;&amp; npm run pagar / entregar / liquidar</code>                |

[← Whitepaper](#) · [Functional documentation](#) · [Code review](#)